

Lesson 24: Introducing Multithreading and Multiprocessing in VEX IQ Programming

Objective:

In this lesson, students will learn the concepts of multithreading and multiprocessing, focusing on how these principles can be applied in VEX IQ programming to control multiple robot tasks simultaneously. By the end of the lesson, students will be able to understand the difference between multithreading and multiprocessing, implement basic multithreading in VEXcode IQ, and manage multiple robot actions effectively.

Lesson Outline

- **Part 1: Introduction to Multithreading and Multiprocessing**
 - **Part 2: Implementing Multithreading in VEX IQ Programming**
 - **Part 3: Testing and Managing Multiple Robot Tasks**
-

Materials Needed:

- VEX IQ Robot with multiple actuators and sensors (e.g., motors, distance sensors, Gyro Sensor)
 - Computers or tablets with VEXcode IQ software
-

Part 1: Introduction to Multithreading and Multiprocessing

1. Understanding Multitasking in Robotics:

- Discuss how robots often need to perform multiple actions simultaneously (e.g., moving while monitoring a sensor).

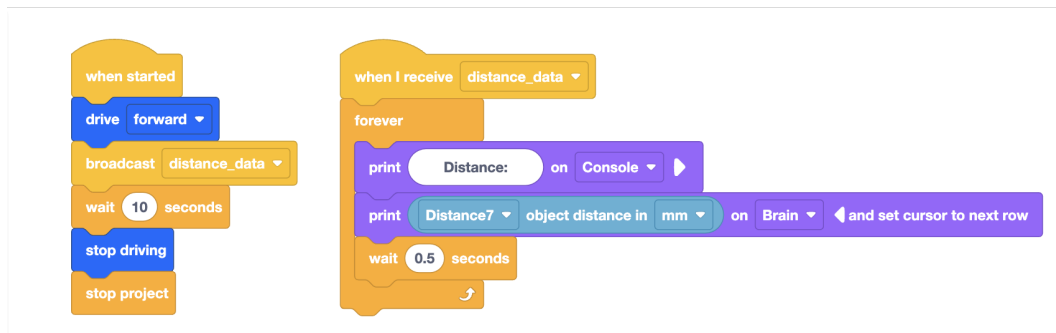
- Define the concept of multitasking and how it improves the robot's efficiency and responsiveness.

2. Multithreading vs. Multiprocessing:

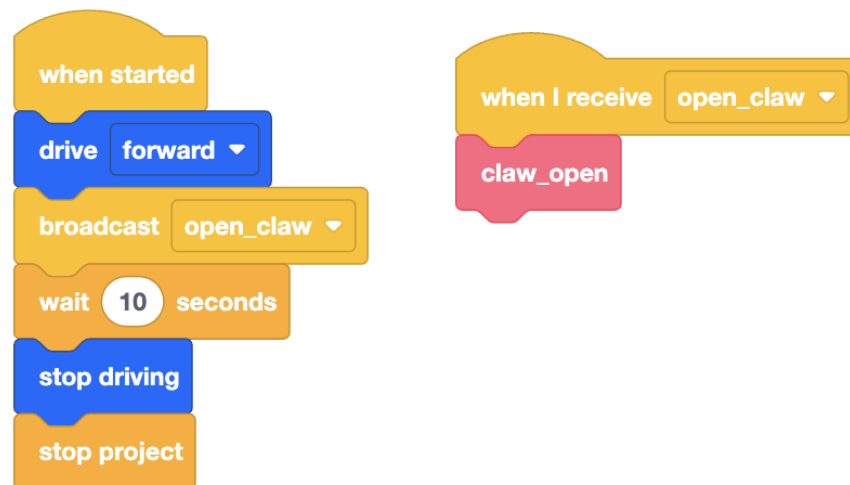
- **Multithreading:** Explain that multithreading involves running multiple tasks (threads) concurrently within a single program. This is like having different tasks share the same resources but switch back and forth quickly.
- **Multiprocessing:** Describe multiprocessing as running separate programs (processes) on different processors or cores. Each process has its own resources.
- Highlight that VEX IQ programming supports multitasking (similar to multithreading) to manage multiple actions at once.

3. Examples of Multithreading in VEX IQ:

- Moving the robot forward while simultaneously reading distance data from a sensor.
- Example code:



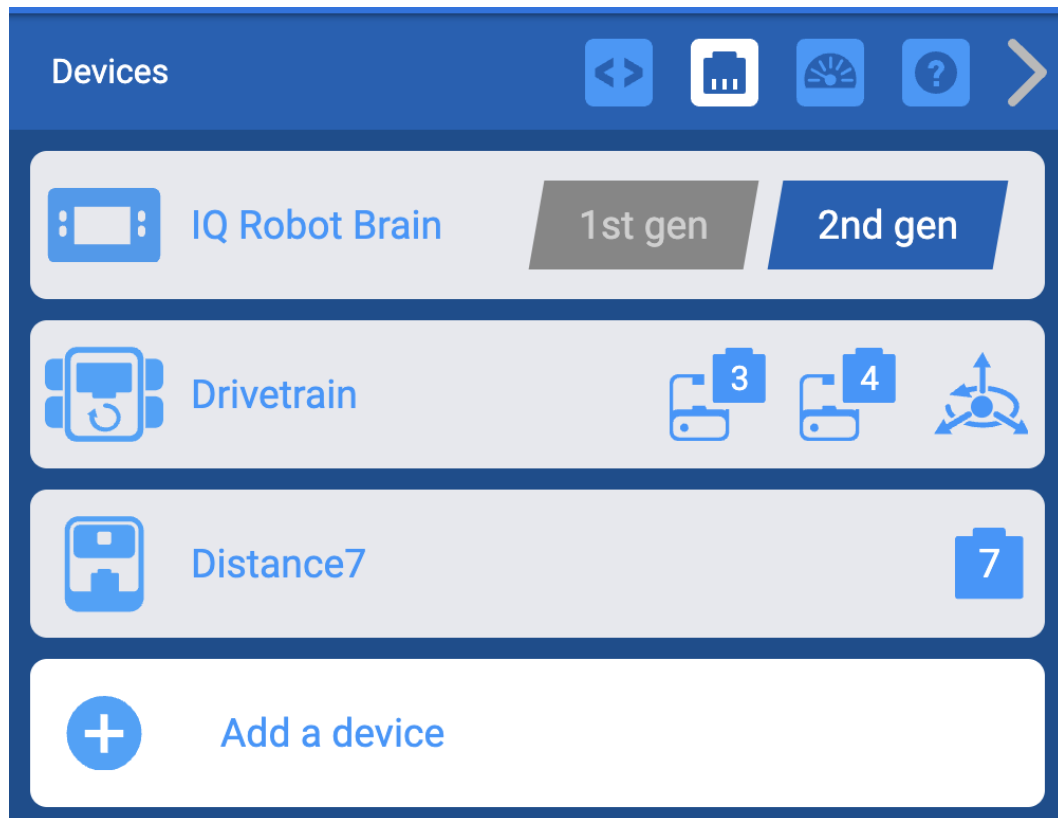
- Operating a claw while the robot is driving forward.
- Example code:



Part 2: Implementing Multithreading in VEX IQ Programming

1. Setup in VEXcode IQ:

- Open VEXcode IQ, create a new project, and configure the robot's components, including motors and sensors.



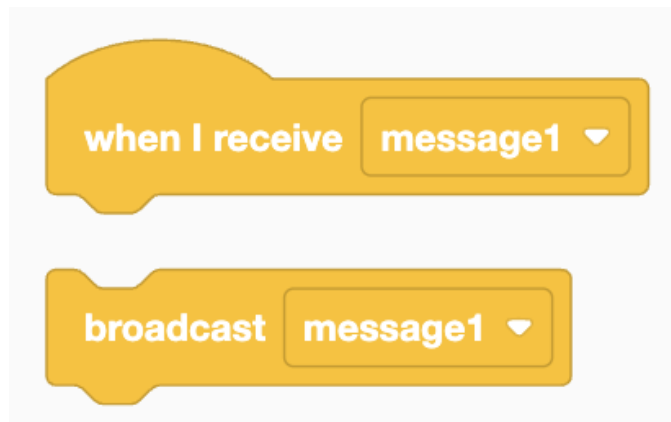
Example of the config on Vexcode IQ

2. Define Tasks for Multithreading:

- Introduce the concept of creating "tasks" in VEXcode IQ that can run concurrently.
- Explain how tasks are like threads and can be used to handle different robot actions at the same time.

3. Creating and Running Tasks:

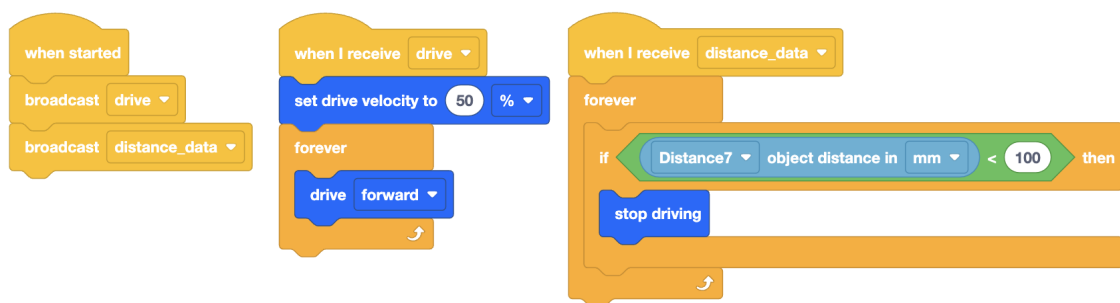
- Use `Broadcast` and `When I receive` blocks to create multiple tasks that the robot can handle concurrently.



The block to start the multiprocessing

- **Task:** Create a task to drive the robot forward while monitoring a sensor.

4. Implement Multithreading Code Example (Pseudocode):



5. Run Initial Test:

- Run the program to observe how the robot performs multiple tasks at the same time.
- Emphasize the importance of keeping tasks synchronized and avoiding conflicts between actions.

Part 3: Challenge

- Integrate the multithreading/multiprocessing concept we learn from this lesson to the last lesson PID Controller program
- Try to program your bot can shoot the ball and then reload. In the meantime, drive the bot backward to intake the balls.
- The ultimate goal: Tackle the 2024-2025 Rapid Relay Autonomous challenge

Lesson Wrap-Up

- **Review Key Concepts:** Differences between multithreading and multiprocessing, how to create and manage tasks in VEX IQ programming, and the benefits of multitasking in robotics.
- **Q&A:** Answer any questions students may have about multithreading or task management.
- **Real-World Applications:** Discuss examples of multitasking in industrial robotics, drone control, and everyday applications (e.g., smartphones managing multiple apps).

This lesson provides a practical introduction to multithreading, demonstrating how to handle multiple robotic actions simultaneously in VEX IQ programming. Students will gain valuable experience in managing complexity, which is fundamental in advanced robotics.